

Image-to-Video Semantic Retrieval via Object Detection

Steven Sagun
New Jersey Institute of Technology
DS 681
Spring 2026

1 Introduction

In this assignment, an application is developed to be able to train a model to identify the various car parts in images and, based on those car parts, find all instances where each part appears in a video. This task involves training an object detector model, frame sampling, indexing, prediction and lookups.

2 Detector choice and configuration

YOLOv12 was chosen as the detector for this assignment, specifically Ultralytics' YOLO12m. There are many object detection models available and many different implementations of YOLO, but the main deciding factor for this choice was the seamless integration of the model with the training dataset. There were no modifications needed to get the model to work with the dataset. Also, the Ultralytics tooling provides a simple interface for training and evaluating the model with the dataset. The configuration and hyperparameters used in the training can be found in Table 1. With these settings, the model achieved a mAP@0.5 of 0.879 on the validation set.

Parameter	Value
Model	YOLO12m
Image size	640
Epochs	100
Batch Size	16
Dropout	0.1
fliplr	0.0

Table 1: Model configuration used during training.

The fliplr parameter played a significant role in the training of the model. This hyperparameter is the probability of applying a horizontal flip to the input image during training. This would be helpful if we wanted to identify the parts of the car irrespective of which side they are on. However, the classes of the dataset include specific left and right labels, e.g., back_left_door and front_right_door. Applying a horizontal flip to the input image without updating the label would

result in the model potentially learning the wrong classes. The default value for this hyperparameter is 0.5, which means that there is a 50% chance of applying a horizontal flip to the input image during training. Setting this hyperparameter to 0.0 ensures that no horizontal flips are applied to the training images.

3 Video sampling strategy

After the model was trained, the input video was downloaded using yt-dlp with a resolution of 480p. The video was renamed to video.mp4 for convenience and is used as the video_id in the parquet file. The video was then sampled at a rate of 1 frame per second (fps) using OpenCV, noting which second of the video the frame appeared at. For each of these frames, the training model predicts which classes were present in the frame, and their corresponding bounding boxes and confidence scores. For each class in each frame, the video_id, the second of the video, the class label, bounding box, and confidence score are stored in a list as an object. This list is then used to create the parquet file using Pandas. The parquet file was then uploaded to Hugging Face which can be viewed at <https://huggingface.co/datasets/Sutiibun/DS681-Assignment2-Detections>. As shown in Figure 1, the model is able to correctly identify the different parts of the car along with the bounding boxes and confidence scores.

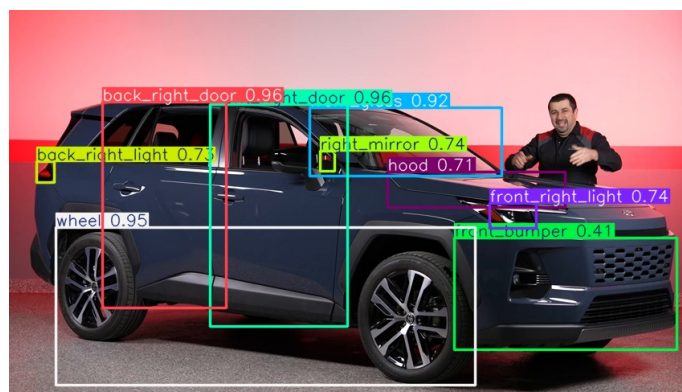


Figure 1: Model predictions at timestamp 0 for the input video.

4 Image-to-video matching logic

At this point, the video has already been processed and we know which classes are present or not present in each second of the video. The parquet file needs to be further processed in order for us to figure out the start and end times for the Youtube URL. The parquet file is read using Pandas as a DataFrame. For each class, we filter the DataFrame such that the class_label equals the class of interest. Then, we sort the results by the timestamp in ascending order. Each contiguous list of timestamps would be considered a section of the video where the class is present. Using this contiguous list, we can derive the start and end times for the Youtube URL.

Timestamps: $\underbrace{0 \ 1 \ 2 \ 3 \ 4}_{\text{Clip 1}} \quad \underbrace{9 \ 10}_{\text{Clip 2}}$

Figure 2: Illustration of how contiguous timestamps are grouped into clips.

As illustrated in Figure 2, if the timestamps that a certain class appear in are 0, 1, 2, 3, 4, 9, and 10, then there are two clips where the class is present. There is a gap between timestamps 4 and 9, where the class is not present. This signifies the end of a clip (timestamp 4) and the start of a new clip (timestamp 9). From this information, we can derive the start and end times for each clip to generate the Youtube URL as illustrated in Figure 3. Using this method, a dictionary is constructed where the keys are the classes and the values are the list of Youtube URLs.

Timestamps: $\underbrace{0 \ 1 \ 2 \ 3 \ 4}_{\text{Clip 1}} \quad \underbrace{9 \ 10}_{\text{Clip 2}}$

↓

<https://www.youtube.com/embed/YcvECxtXoxQ?start=0&end=4>
<https://www.youtube.com/embed/YcvECxtXoxQ?start=9&end=10>

Figure 3: Illustration of how contiguous timestamps determine the start and end times.

Finally, with this setup, given any query image, the model can predict the classes for that image and simply retrieve the links based on those classes from the constructed dictionary. A sample output can be found in Appendix A

5 Failure cases and limitations

As mentioned in this report, there were a few failure cases and limitations that were encountered. When training without setting the `fliplr` parameters, it significantly reduced performance, as the training images were being flipped 50% of the time while also not adjusting the labels appropriately. The obvious effect was the misclassification of the left and right parts. One thing to note is that for the "wheel" class, the bounding box would sometimes span both wheels of a side instead of having two bounding boxes for each. Another limitation, while not related to the assignment, was the Youtube link itself. Opening the link directly gave an error on playback. The link had to be played from within an `iframe`. As a workaround, I created a Github Page to load the video with the specified start and end timestamps. Refer to Appendix A under the Playback URL column for an example.

A Sample output

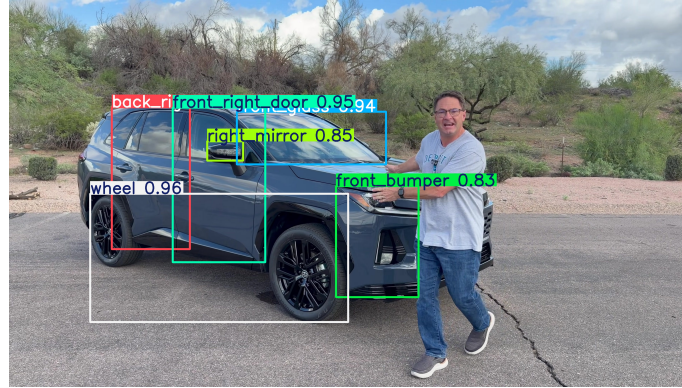


Figure 4: Model predictions for a query image.

Class	Class Name	Youtube URL	Playback URL
6.0	back_right_door	https://www.youtube.com/embed/YcvECxtXoxQ?start=1330&end=1336	Link
8.0	front_bumper	https://www.youtube.com/embed/YcvECxtXoxQ?start=3&end=8	Link
10.0	front_glass	https://www.youtube.com/embed/YcvECxtXoxQ?start=1119&end=1129	Link
14.0	front_right_door	https://www.youtube.com/embed/YcvECxtXoxQ?start=1529&end=1542	Link
19.0	right_mirror	https://www.youtube.com/embed/YcvECxtXoxQ?start=0&end=10	Link
22.0	wheel	https://www.youtube.com/embed/YcvECxtXoxQ?start=1212&end=1215	Link

Table 2: A few results with their corresponding clips. Refer to main.py for the full output.